02 Manoj

Background and proposed system architecture

Fuzzy-Logic-Based QoS Control in oneM2M (Zyane & Et-Tousy, FiCloud 2026)

Your job

In about **4 minutes**: explain oneM2M just enough, walk the architecture (paper Fig. 1) and the workflow (Fig. 2), contrast edge and cloud, and explain the MAPE-K loop. End by pointing at the "Plan" box: that is where Ashutosh takes over. You do *not* explain how the fuzzy controller computes anything.

Your slides

Slides 10–17: part opener, oneM2M, Fig. 1, three layers, edge vs. cloud, workflow, MAPE-K, hand-off.

Opening line

"Thanks Raj. Before the clever part, let me show you the plumbing: what oneM2M is, and where the controller sits."

Hand-off line

"Everything I've shown is plumbing. The interesting question is how alpha is chosen. Ashutosh?"

Glossary

CSE	Common Services Entity: the oneM2M server itself
AE	Application Entity: an app registered on the CSE
CNT / CIN	container / contentInstance: a folder / one data reading
Mca, Mcc	app↔CSE link, CSE↔CSE link
Mobius	open-source oneM2M CSE used in the paper (IoT Ocean)
MAPE-K	Monitor, Analyze, Plan, Execute, shared Knowledge
α	share of traffic sent to the cloud, 0 to 1

Numbers you should know

Edge node: Ubuntu 22.04, 4 vCPU, 4 GiB RAM. Cloud: AWS EC2 t2.medium (2 vCPU, 4 GiB). Loop runs every 5 s. Messages are 10–50 KB in general, 1 KB in the experiment.

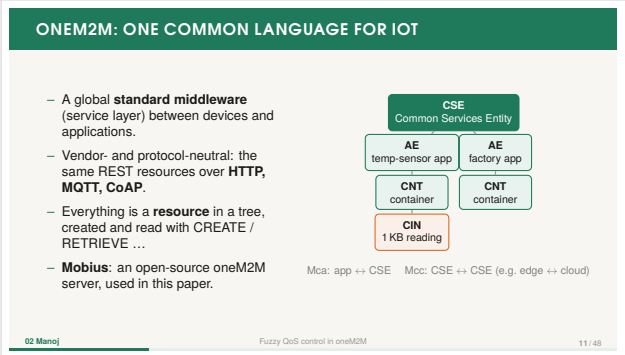
Slide 10 ~10 s



Your part opener: green, “02 Manoj”.

Say: “Thanks Raj. Before the clever part, let me show you the plumbing: what oneM2M is, and where the controller sits.”

Slide 11 ~40 s



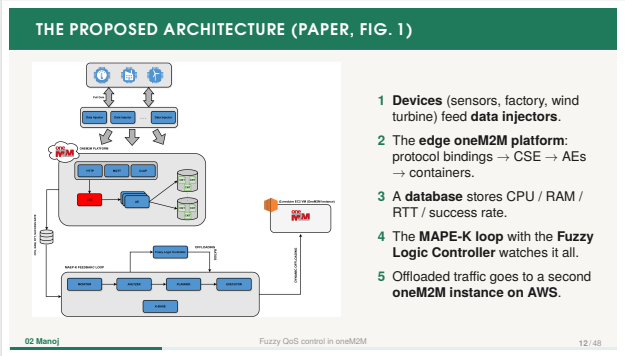
oneM2M in one slide: bullets on the left, resource tree CSE → AE → CNT → CIN on the right.

Say: “oneM2M is a global standard for IoT middleware. It sits between devices and applications so that devices from different vendors, speaking different protocols, HTTP, MQTT or CoAP, all look the same to the app.”

Say: “Everything is a resource in a tree. At the top is the CSE, the oneM2M server. Applications register as AEs. Each AE has containers, and every sensor reading is a contentInstance, in this paper a 1 KB reading. Devices just send CREATE requests to add readings.”

Say: “The server used here is Mobius, an open-source oneM2M implementation. The link between two oneM2M servers, say edge and cloud, is called Mcc. That is the link offloaded traffic travels over.”

Slide 12 ~40 s



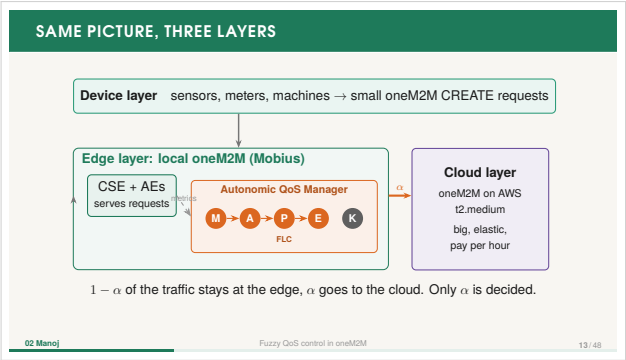
The paper’s own architecture figure, with five numbered steps.

Say: “This is the paper’s Figure 1. Top: devices, a thermometer, a factory, a wind turbine, feeding data injectors. Middle: the edge oneM2M platform with its protocol bindings, the CSE in red, the application entities and their containers.”

Say: “Bottom: the self-management loop, Monitor, Analyzer, Planner, Executor, and a Knowledge base, plus the Fuzzy Logic Controller. On the right, a second oneM2M server on Amazon, which receives whatever traffic gets offloaded.”

Tip: The figure is busy. Point at regions with your hand rather than reading labels. The next slide redraws it simply.

Slide 13 ~35 s



Simplified redraw: device layer, edge layer with the Autonomic QoS Manager (M-A-P-E-K), cloud layer; orange arrow α.

Say: “Here is the same thing in three layers. Devices at the top send requests. The edge layer runs Mobius, and next to it the Autonomic QoS Manager, the M-A-P-E-K loop. The cloud layer is another Mobius on AWS.”

Say: “The key point: the only thing the system decides is this orange arrow, alpha. One minus alpha of the traffic stays at the edge, alpha goes to the cloud.”

Slide 14 ~30 s

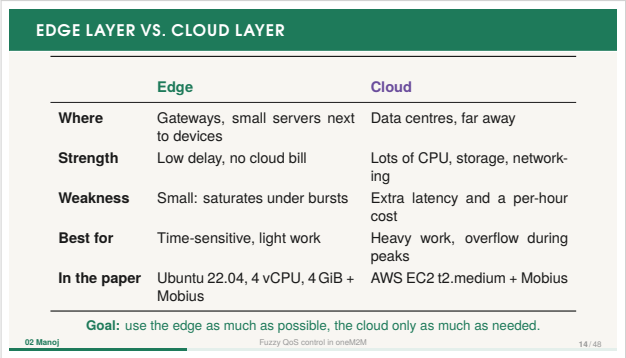
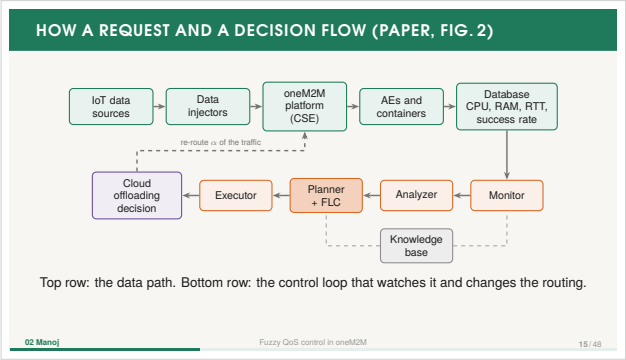


Table comparing edge and cloud: where, strength, weakness, best for, what was used.

Say: “Why not just use one or the other? The edge is close and has no bill, but it is small and saturates under bursts. The cloud has plenty of capacity, but adds latency and charges per hour.”

Say: “So the goal is to use the edge as much as possible, and the cloud only as much as needed. In the paper the edge is a 4-core Ubuntu box and the cloud is an AWS t2.medium.”

Slide 15 ~40 s

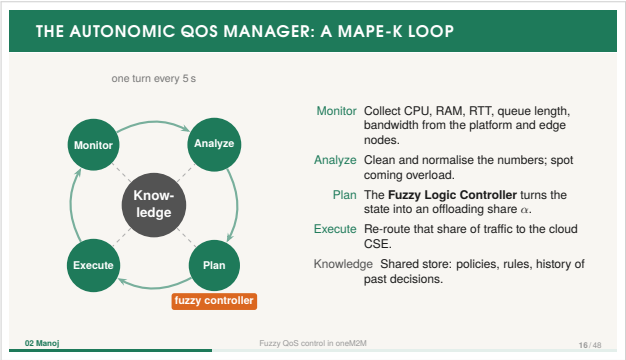


Workflow (redraw of paper Fig. 2). Top row: data path. Bottom row: control loop, Monitor → Analyzer → Planner+FLC → Executor → decision.

Say: "This is the step-by-step flow, the paper's Figure 2. Along the top, a reading travels from the device, through the injectors, into the oneM2M platform, into an application's container, and its performance numbers, CPU, RAM, RTT, success, land in a database."

Say: "Along the bottom, the control loop reads that database. Monitor collects, Analyzer cleans, the Planner with the fuzzy controller decides, the Executor acts, and the decision goes back up to re-route alpha of the traffic. The knowledge base feeds all of them."

Slide 16 ~45 s



The MAPE-K circle with Knowledge in the middle; descriptions of each stage on the right.

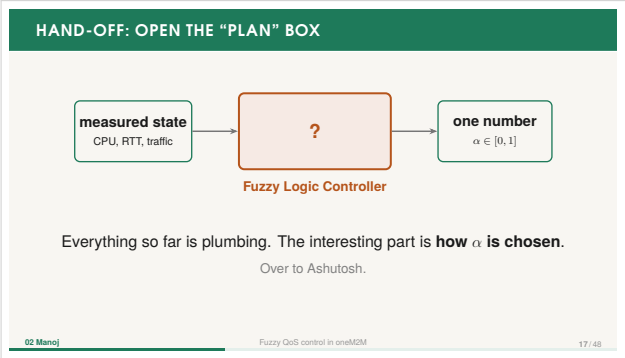
Say: "MAPE-K is a standard pattern for self-managing systems, originally from IBM's autonomic computing work. One turn of this loop runs every 5 seconds."

Say: "Monitor gathers CPU, RAM, round-trip time, queue length and bandwidth. Analyze normalises them and spots overload coming. Plan is where the fuzzy controller turns the state into alpha. Execute re-routes that share of traffic to the cloud. Knowledge stores the rules, policies and history, so later cycles can learn from earlier ones."

Q: *Does the Knowledge base actually learn?*

A: In this paper it stores rules and history, but the rules are fixed during the experiments. Learning the rules is listed as future work.

Slide 17 ~15 s



Measured state → “?” box → one number α.

Say: “So: three numbers come in, one number comes out. Everything I’ve shown is plumbing. The interesting question is how alpha is chosen. Ashutosh?”

If you get questions during Q&A

Q: *Why put the controller inside oneM2M instead of in a separate load balancer?*

A: Then any oneM2M application gets the QoS behaviour without changes, and the controller can see oneM2M-level metrics directly. The paper claims it is the only surveyed approach that does this end to end.

Q: *How does the offloaded traffic reach the cloud?*

A: Over the oneM2M CSE-to-CSE link (Mcc) to a second Mobius instance on AWS; the Executor changes the routing.

Q: *Why 5 seconds?*

A: Fast enough to follow bursts, slow enough that analysis costs under 3 % of one CPU core. Ashutosh has the configuration slide.